oasis-wp3 documentation

Contenu

Contents:

- Interface principale du réseau SUMO
- Fonctionnalités liées aux composants du réseau SUMO
- Fonctionnalités liées aux véhicules du réseau SUMO

Interface principale du réseau SUMO

class oasisPrototype.oasisPrototype.OasisPrototype(*sumoBin: str = 'sumo-gui', sumoCfg: str = 'prototype.sumocfg', sumoContainers: str = 'containers.add.xml', sumoParkings: str = 'parkings.add.xml', sumoNet: str = 'network.net.xml', delay: str = '50'*)

Bases : `object`

Paramètres:

- **sumoBin** – Chemin absolu vers le binaire SUMO ou alias exécutable (par exemple « sumo-gui » ou « sumo »)
- **sumoCfg** – Chemin vers le fichier de configuration réseau « <nom>.sumocfg », utilisé comme point d'entrée de la simulation SUMO
- **sumoContainers** – Chemin vers le fichier contenant les containers de la simulation « <nom>.add.xml »
- **sumoParkings** – Chemin vers le fichier contenant les parkings de la simulation « <nom>.add.xml »
- **sumoNet** – Chemin vers le fichier de la modélisation du réseau SUMO « <nom>.net.xml »
- **delay** – Délai en millisecondes appliqué entre chaque step de simulation, permettant de ralentir ou synchroniser l'exécution

Exemple d'utilisation:

```
from oasisPrototype import OasisPrototype
oasis = OasisPrototype(sumoCfg="./sumo/prototype.sumocfg",
                       sumoContainers="./sumo/additional/containers.add.xml",
                       sumoParkings="./sumo/additional/parkings.add.xml",
                       sumoNet="./sumo/réseau_prototype.net.xml")
oasis.runOasis() # pour lancer la simulation dans un thread
```

runOasis()

Lance la simulation SUMO dans un thread séparé en utilisant TraCI

> **Note**
>
> En mode développement (devMode=True), la simulation est lancée directement dans le thread courant (appel bloquant) En mode production, la simulation est lancée dans un thread dédié

Fonctionnalités liées aux composants du réseau SUMO

class oasisPrototype.network.oasisNetwork.OasisNetwork(*_addThreadQueue*, *sumoNet: str*, *sumoContainers: str*, *sumoParkings: str*)

Bases : `object`

closeEdge(*edgeID: str*, *classesDisallowed=list[str]*)

Ferme un edge dans la simulation SUMO en interdisant une ou plusieurs classes de véhicules

Paramètres:

- **edgeID** (*str*) – Identifiant de l'edge à fermer
- **classesDisallowed** (*list[str]*) – Liste des classes de véhicules à interdire

Exemple d'utilisation:

```
oasis.network.closeEdge(edgeID="E06to05", classesDisallowed=["truck"])
```

createRide(*edgesAndContainer: dict*) → dict

Créer et retourne un json de trajet pour l'ajout d'un camion dans une simualtion SUMO

Paramètres:

edgesAndContainer (*dict*) – Structure des edges et id de containers

Renvoie:

json contenant les id des edges et position par container:

```
 {
    "E07to06": [],
    "E06to05": [
        {
            "pos": 403.97,
            "duration": 3.00,
        }
    ],
    "E05to21": [],
    "E21to17": [],
    "E17to06": [],
    "E06to07": [],
    "E07to08": []
}
```

Type renvoyé:

tuple[str, dict]

Exemple d'utilisation:

```
RIDE = {
            "E07to06": [],
            "E06to05": [
                "c_11"
            ],
            "E05to21": [],
            "E21to17": [],
            "E17to06": [],
            "E06to07": [],
            "E07to08": []
        }

trip = oasis.network.createRide(edgesAndContainer=RIDE)
```

getAllNetwork(*offline: bool = True*) → dict

Retourne un dictionnaire représentant le réseau complet de la simulation

Paramètres:

offline (*bool*) – run de la méthode hors simulation SUMO

Note

Contient les junctions, edges, lanes et conteneurs connus au moment de l'appel Si un fichier de conteneurs est configuré mais non parsé, il est parsé automatiquement

Renvoie:

Structure du réseau:

```
{
    "edges": {
        "E21to18": {
            "lane": "E21to18_0",
            "junctions": {"from": "J21", "to": "J18"},
            "containers": {...}
        },
        ...
    }
}
```

Type renvoyé:

dict[str, dict]

Exemple d'utilisation:

```
oasis.network.getAllNetwork()
```

getContainerById(*containerID: str*) → dict

Retourne les informations d'un container par son ID

Paramètres:

containerID (*str*) – id du container

Renvoie:

dict[str, str]:

```
{"pos": "221.29", "edgeID" : "E06to05"}
```

Exemple d'utilisation:

```
oasis.network.getContainerById(containerID="c_12")
```

getContainers() → dict

Retourne les conteneurs de la simulation si le fichier XML a été fourni et parsé

Note

Si l'attribut interne containers est vide et qu'un chemin de fichier XML est défini, le fichier est parsé puis les conteneurs sont renvoyés

Si containers contient déjà des données, ces données sont renvoyées directement.

Sinon, un dictionnaire d'information est renvoyé pour indiquer l'absence de conteneurs ou l'absence de fichier XML

Renvoie:

Dictionnaire des conteneurs indexé par identifiant:

```
{
    "c_1": {"id_edge": "E03to02"},
    "c_10": {"id_edge": "E06to05"},
    ...
}
```

Si aucun conteneur n'est disponible:

```
{"info": "Aucun container existant ou fichier xml non transmis"}
```

Type renvoyé:

dict[str, dict] | dict

Exemple d'utilisation:

```
oasis.network.getContainers()
```

`getContainersByEdge()` → `dict`

Retourne le(s) conteneur(s) présent dans chaque edge

Renvoie:

dict[str, dict]:

```
{
    "E03to02": ["c_1"],
    "E06to05": ["c_10", "c_11", "c_12", "c_13"],
    "E09to07": ["c_14", "c_15", "c_16"]
    ...
}
```

Exemple d'utilisation:

```
oasis.network.getContainersByEdge()
```

getEdges(*offline: bool = True*) → tuple

Retourne les identifiants de tous les edges du réseau

Paramètres:

offline (*bool*) – run de la méthode hors simulation SUMO

> **Note**
>
> Filtre la liste fournie par TraCI pour ne conserver que les edges respectant la convention de nommage utilisée (préfixe « E »)

Renvoie:

Tuple d'identifiants d'edges:

```
("E21to18", "E10to5", ...)
```

Tuple vide si aucune edge ne correspond ou en cas d'erreur

Type renvoyé:

tuple[str, …]

Exemple d'utilisation:

```
oasis.network.getEdges()
```

getJonctions(*offline: bool = True*) → tuple

Retourne les identifiants de toutes les junctions du réseau

Paramètres:

offline (*bool*) – run de la méthode hors simulation SUMO

> **Note**
>
> Filtre la liste fournie par TraCI pour ne conserver que les junctions respectant la convention de nommage utilisée (préfixe « J »)

Renvoie:

Tuple d'identifiants de junctions:

```
("J01", "J21", ...)
```

Tuple vide si aucune junction ne correspond ou en cas d'erreur

Type renvoyé:

tuple[str, …]

Exemple d'utilisation:

```
oasis.network.getJonctions()
```

getLanes(*offline: bool = True*) → tuple

Retourne les identifiants de toutes les lanes du réseau

Paramètres:

offline (*bool*) – run de la méthode hors simulation SUMO

Note

Filtre la liste fournie par TraCI pour ne conserver que les lanes respectant la convention de nommage utilisée (préfixe « E »)

Convention de nommage des lanes : E21to18_0 (le suffixe « _0 » est généré automatiquement par SUMO)

Renvoie:

Tuple d'identifiants des lanes:

```
("E21to18_0", "E10to5_0", ...)
```

Tuple vide si aucune lane ne correspond ou en cas d'erreur

Type renvoyé:

tuple[str, …]

Exemple d'utilisation:

```
oasis.network.getLanes()
```

getParkings(*offline: bool = True*) → tuple

Retourne les IDs des parkings du réseau

Paramètres:

offline (*bool*) – run de la méthode hors simulation SUMO

Renvoie:

Tuple d'identifiants des parkings:

```
("pa_1", "pa_2", ...)
```

Type renvoyé:

tuple[str, …]

Fonctionnalités liées aux véhicules du réseau SUMO

class oasisPrototype.network.oasisNetworkVehicules.OasisNetworkVehicules(*_addThreadQueue*)

Bases : `object`

addCamion(*vehID: str, parkingID: str, typeID: str = 'truck', depart: str = 'now', trip: dict[str, float] | None = None*)

Ajoute un camion à la simulation SUMO et configure sa route et ses arrêts

Paramètres:

- **vehID** (*str*) – Identifiant du camion à créer
- **parkingID** (*str*) – Identifiant de la zone de parking où le camion doit s'arrêter à la fin
- **typeID** (*str, optional*) – Type de véhicule SUMO, par défaut « truck »
- **depart** (*str, optional*) – Moment de départ, par défaut « now »
- **trip** (*dict* | *None, optional*) – Structure des edges et arrêts

Exemple d'utilisation:

```
TRIP = {
        "E07to06": [],
        "E06to05": [
            {
                "pos": 403.97,
                "duration": 3.00,
            }
        ],
        "E05to21": [],
        "E21to17": [],
        "E17to06": [],
        "E06to07": [],
        "E07to08": []
    }
oasis.networkVehicules.addCamion(vehID="truck_1",
                                 edgesAndStops=TRIP,
                                 parkingID="pa_1")
```

`getCamionDataByID(vehID: str) → list`

Retourne les informations suivantes pour un camion dans la simulation :

- fuel : consommation de carburant total (en mg/s)
- parking : identifiant du parking assigné au camion
- parkingStatus : True si le camion est actuellement garé

Les valeurs reflètent l'état du véhicule à l'instant T de la simulation.

Paramètres:

vehID (*str*) – Identifiant du camion

> **Note**
>
> Retourne une liste vide si le camion n'existe pas

Renvoie:

données d'un camion:

```
{"fuel": 32222.0670, "parking": "pa_3", "parkingStatus": True}
```

Type renvoyé:

list

Exemple d'utilisation:

```
oasis.networkVehicules.getCamionDataByID(vehID="truck_1")
```

`getDistanceParcourue(vehID: str)`

Retourne la distance parcourue par un véhicule depuis son démarrage

> **Note**
>
> Valeur renvoyée uniquement si le véhicule est présent dans la simulation Renvoie 0.0 si le véhicule est absent ou si la distance n'est pas disponible

Paramètres:

vehID (*str*) – Identifiant du véhicule

Renvoie:

Distance parcourue en mètres, 0.0 si indisponible

Type renvoyé:

float

Exemple d'utilisation:

```
oasis.networkVehicules.getDistanceParcourue(vehID="truck_1")
```

getStatusVehiculeByID(*vehID: str*) → bool

Indique si un véhicule est présent (actif) dans la simulation SUMO

> **Note**
>
> Renvoie True si l'identifiant du véhicule figure dans la liste des véhicules actifs fournie par TraCI, False sinon

Paramètres:

vehID (*str*) – Identifiant du véhicule à vérifier

Renvoie:

True si le véhicule est présent dans la simulation, False sinon

Type renvoyé:

bool

Exemple d'utilisation:

```
oasis.networkVehicules.getStatusVehiculeByID(vehID="truck_1")
```

getTempsDeTrajetCamion(*vehID: str*) → str

Retourne le temps de trajet total d'un camion depuis son démarrage

> **Note**
>
> Valeur disponible uniquement lorsque le camion se gare au dépôt Renvoie « 0 » si le camion est encore en trajet ou s'il n'est pas présent dans la simulation

Paramètres:

vehID (*str*) – Identifiant du camion

Renvoie:

Temps de trajet sous forme de chaîne:

```
"123.45"
"0" si indisponible
```

Type renvoyé:

str

Exemple d'utilisation:

```
oasis.networkVehicules.getTempsDeTrajet(vehID="truck_1")
```

getVehiculesData() → dict

Retourne les données des véhicules présents dans la simulation à l'instant T

Note

Renvoie un dictionnaire vide si aucun véhicule n'est présent ou si une erreur survient Les distances et positions sont formatées en chaîne avec deux décimales

Renvoie:

Dictionnaire dont les clés sont les identifiants des véhicules:

```
{
    "0": {
        "edgeID": "E16to09",
        "laneID": "E16to09_0",
        "lanePosition(m)": "173.51",
        "distance(m)": "168.41",
        "timeSimu(s)": 13.0,
        "routes": ["E16to09", "E09to10", "E10to15", "E15to18",
                "E18to29", "E19to20", "E20to02", "E02to03"]
    }
}
```

Type renvoyé:

dict[str, dict]

Exemple d'utilisation:

```
oasis.networkVehicules.getVehiculesData()
```

updateCamionFuel(*parkingsIDs: str*)

Met à jour et vérifie si chaque camion de la simulation est toujours actif ou non (dans un parking ou non)

Paramètres:

parkingsIDs (*list[str]*) – Identifiants des parkings